

Testing with ExUnit — A Readable Companion to the Elixir Docs

Chapter 11 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why testing concurrent code needs its own habits

ExUnit makes tests easy to write and fast to run: `async: true` runs test modules in parallel, `assert_receive` checks messages, `start_supervised!` manages processes, and doctests turn documentation into tests. The general principles (test the unhappy paths, use properties, don't restate the implementation) are in the Erlang book, chapter 8.

Elixir-specific trouble comes from concurrency inside the test suite itself. Tests that pass one at a time fail when run together, or pass today and fail on a loaded CI machine. The culprits are almost always shared state, fixed timeouts that are too short or too long, and processes that outlive or escape the test that started them.

2. The building blocks

Tool	Meaning
<code>use ExUnit.Case, async: true</code>	This module's tests run concurrently with other async modules; tests in one module still run one at a time
<code>async: true, group: :name (1.18+)</code>	Modules in the same group never run concurrently
<code>setup / setup_all</code>	Per test (in the test process) / once per module (in a separate process)
<code>start_supervised!/1</code>	Start a process under the test's supervisor; stopped when the test ends
<code>assert_receive pattern, timeout</code>	Wait for a message; default timeout 100 ms
<code>refute_receive pattern, timeout</code>	Check no message arrives within the timeout; also 100 ms by default
<code>capture_log/1, @tag capture_log: true</code>	Capture log output
<code>doctest Mod</code>	Run the <code>iex></code> examples in <code>Mod</code> 's documentation
Test timeout	60 seconds per test by default

For mocking, the standard library is Mox. Its rules: mocks are defined from behaviours (chapter 8), expectations are private to the test process by default, and child processes need `allow/3`. Processes started with `Task` are allowed automatically through `$callers`. Global mode is "incompatible with `async: true`."

3. How it actually works

async means other modules run at the same time

With `async: true`, your module runs at the same time as other async modules. Anything global is shared across all of them: registered names, the application environment, ETS tables with fixed names, Logger configuration, the file system, a real database without a sandbox. The docs: async “should be enabled only if tests do not change any global state.”

Short default timeouts cut both ways

`assert_receive` waits only 100 ms by default, which is too short for anything that crosses a slow CI machine. `refute_receive` also waits only 100 ms, so it can pass for a message that arrives 200 ms later. Both kinds of error are silent in fast local runs.

The test process is special

Processes started with `start_link` inside a test are linked to the test process and die with it, while `start_supervised!/1` stops them in order and waits. Code running in *other* processes (a GenServer under test) doesn't carry the test's Mox expectations unless allowed.

4. Plausible but wrong

4.1 Async tests that share a named process

```
defmodule CacheTest do
  use ExUnit.Case, async: true
  test "caches values" do
    {:ok, _} = MyApp.Cache.start_link([]) # registers name: MyApp.Cache
    ...
  end
end
```

Run alongside another async module that starts the same named server, and one of them fails with `{:error, {:already_started, pid}}`, depending on scheduling. In a test with three such modules, two failed (verified). Fix: accept a `:name` option (chapter 4, section 4.3) and start a uniquely named instance per test, with `start_supervised!({MyApp.Cache, name: unique_name})`, or make the module `async: false`.

4.2 Async tests that change the application environment

```
test "strict mode rejects bad input" do
  Application.put_env(:my_app, :mode, :strict)
  assert {:error, _} = MyApp.validate(bad_input)
end
```

The application environment is global. Another async test setting `:mode` to `:lenient` in the meantime makes this fail with the other test's value (verified: left: `:lenient`, right: `:strict`). Fix: pass options explicitly to the code under test (chapter 10, section 4.5). Where that's impossible, run the module with `async: false` and restore the value in `on_exit/1`.

4.3 refute_receive that proves nothing

```
test "does not email unconfirmed users" do
  Signup.register(unconfirmed_user)
  refute_receive {:email_sent, _}
end
```

`refute_receive` waits 100 ms. If the email is sent 300 ms later by a background job, the test still passes, and the bug ships (verified: the message arrived after `refute_receive` had already passed). Fix: synchronise on a point after which the email would already have been sent, such as a `call` to the process that sends it or an explicit completion message, then `refute_received` (no wait) or `refute_receive` with a justified timeout.

4.4 assert_receive that fails under load

```
Worker.process_async(job)
assert_receive {:done, ^job}
```

The work takes 300 ms on a busy CI runner and the default 100 ms timeout fails the test (verified: “no matching message after 100ms”). It passes locally, so it looks flaky rather than wrong. Fix: give `assert_receive` a timeout that reflects the operation (`assert_receive {:done, ^job}, 2_000`); a long timeout on `assert_receive` costs nothing when the message arrives quickly.

4.5 Sleeping instead of synchronising

```
Counter.increment_async(pid)
Process.sleep(50)
assert Counter.get(pid) == 1
```

Too short on a slow machine, wasted time on a fast one. Fix: the call is the synchronisation. Because messages from one process arrive in order, `Counter.get/1` (a call) is handled after the cast (Erlang book, chapter 8, section 3). Drop the sleep.

4.6 A mock called from another process

```
test "fetches rates" do
  expect(RatesMock, :fetch, fn -> {:ok, %{usd: 1.0}} end)
  {:ok, pid} = start_supervised(RateServer)
  assert RateServer.current(pid) == %{usd: 1.0}
end
```

`RateServer` calls `RateMock.fetch/0` from its own process, which is not the test process and not a `Task`, so `Mox` raises an unexpected call error there. The server crashes, and the failure appears as a confusing exit. Fix: `Mox.allow(RateMock, self(), pid)` after starting the server, or inject the implementation into the server and test its logic as plain functions.

4.7 Doctests with non-deterministic output

```
@doc """
    iex> MyApp.Session.new("ada")
    %MyApp.Session{user: "ada", id: "4f9c...", started_at: ~U[2026-09-26 10:00:00Z]}
    """
```

The id and timestamp differ on every run, and so do pids, references and the order of large maps. The docs also warn that doctests don't capture side effects and run without a sandbox. Fix: show deterministic parts only (`iex> MyApp.Session.new("ada").user / "ada"`), and keep effectful behaviour in ordinary tests.

5. Review checklist and sources

- ☐ `async: true` modules touch no global state: no fixed names, application env, named ETS tables or shared files
- ☐ Processes in tests start with `start_supervised!/1` and unique names
- ☐ `assert_receive` timeouts reflect the real operation; `refute_receive` follows an explicit synchronisation point
- ☐ No `Process.sleep/1` used to wait for work
- ☐ Mocks are behaviour-based; processes that use them are allowed (`Mox.allow/3`) or receive injected implementations
- ☐ Doctests are deterministic and free of side effects
- ☐ Expected values come from requirements, not from running the implementation (Erlang book, chapter 8)

Sources

- ExUnit.Case: https://hexdocs.pm/ex_unit/ExUnit.Case.html
- ExUnit.Assertions: https://hexdocs.pm/ex_unit/ExUnit.Assertions.html
- ExUnit.DocTest: https://hexdocs.pm/ex_unit/ExUnit.DocTest.html
- Mox: <https://hexdocs.pm/mox/Mox.html>
- Erlang book, chapter 8 (Testing): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>